
Tutorial: Inicialización y Arquitectura Clásica para Proyectos Wear OS

Este módulo guía en la creación e instrumentación desde cero de un entorno de software dedicado exclusivamente a dispositivos corporales (*wearables*), aislando las dependencias de telefonía móvil y optimizando el empaquetado para pantallas de baja escala.

1. Fundamentos Teóricos y Conceptos de la Plataforma `wear_plus`

Al aislar el desarrollo de un *smartwatch* en un repositorio independiente, los ingenieros deben comprender cómo interactúa la máquina virtual de Flutter con los hilos nativos de un sistema operativo embebido de recursos restringidos:

- **WatchShape (Geometría del Lienzo):** Es el nodo raíz de hardware que expone la capa nativa. Intercepta los controladores de pantalla del dispositivo para devolver un enumerador: `WearShape.round` (pantallas circulares) o `WearShape.square` (pantallas cuadradas). Su función principal en ingeniería es servir de base para algoritmos de layout adaptativo.
- **AmbientMode (Ciclo de Vida Energético):** A diferencia de un teléfono que se apaga por completo al bloquearse, el reloj alterna constantemente entre el modo activo y el modo ambiente. Cuando la pantalla entra en bajo consumo (`WearMode.ambient`), el hardware deshabilita el rasterizado de animaciones y reduce drásticamente la tasa de refresco del procesador.
- **Aislamiento de Dependencias:** Al construir un proyecto dedicado, el compilador (Gradle) no arrastra librerías del teléfono. Esto optimiza el peso final del archivo binario (APK), algo crítico considerando que el almacenamiento interno de un reloj suele ser una fracción del de un smartphone.

2. Configuración del Entorno (Plataforma Nativa)

Para que el compilador de Android sepa que este APK no debe ejecutarse en un teléfono convencional, es obligatorio modificar la firma del hardware en el manifiesto nativo.

En Android (`android/app/src/main/AndroidManifest.xml`)

Abre el archivo de configuración nativa y añade la siguiente directiva de hardware justo arriba de la etiqueta `<application>`:

```
<uses-feature android:name="android.hardware.type.watch" />
```

Posteriormente, localiza la etiqueta `<activity>` de la aplicación y asegúrate de agregar la regla de no-redimensionamiento para evitar deformaciones en pantallas circulares:

```
android:resizeableActivity="false"
```

📁 Estructura de Carpetas del Proyecto Separado

Al ser un proyecto dedicado de una sola característica (*Standalone Feature*), la arquitectura de la carpeta `lib/` se simplifica para centrarse en el control directo del lienzo del reloj:

```
mi_proyecto_wearable/  
├─ pubspec.yaml  
└─ lib/  
    ├─ main.dart          <-- Punto de entrada de la máquina virtual  
    ├─ app.dart           <-- Orquestador global de MaterialApp y Temas  
    └─ screens/  
        └─ watch_hello_screen.dart <-- Vista adaptativa y control de hardware
```

Configuración del Archivo `pubspec.yaml`

Este es el archivo de manifiesto de dependencias limpio. Esta estructura asegura la compatibilidad absoluta con Gradle moderno y evitar conflictos de *namespaces*:

```
name: mi_proyecto_wearable  
description: "Proyecto independiente y dedicado para laboratorios Wear OS."  
publish_to: 'none'  
version: 1.0.0+1  
  
environment:  
  sdk: '>=3.0.0 <4.0.0'  
  
dependencies:  
  flutter:  
    sdk: flutter  
  wear_plus: ^2.1.1 # Bifurcación comunitaria moderna compatible con Gradle 8+
```

3. Código Fuente Completo y Comentado

Archivo 1: `lib/main.dart`

```
import 'package:flutter/material.dart';  
import 'app.dart';  
  
void main() {  
  // Asegura el correcto enlace de los canales de plataforma nativos de Android
```

```
WidgetsFlutterBinding.ensureInitialized();
runApp(const WearableApp());
}
```

Archivo 2: lib/app.dart

```
import 'package:flutter/material.dart';
import 'screens/watch_hello_screen.dart';

class WearableApp extends StatelessWidget {
  const WearableApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Wear OS Lab',
      // Remoción obligatoria del banner para maximizar los píxeles útiles en el
      reloj
      debugShowCheckedModeBanner: false,

      // OPTIMIZACIÓN ENERGÉTICA: Forzamos tema oscuro global con fondo negro puro
      (#000000)
      // para asegurar el apagado físico de píxeles en pantallas AMOLED/OLED.
      theme: ThemeData(
        brightness: Brightness.dark,
        scaffoldBackgroundColor: Colors.black,
        useMaterial3: true,
      ),
      home: const WatchHelloScreen(),
    );
  }
}
```

Archivo 3: lib/screens/watch_hello_screen.dart

```
import 'package:flutter/material.dart';
import 'package:wear_plus/wear_plus.dart';

class WatchHelloScreen extends StatelessWidget {
  const WatchHelloScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. Inyección del listener geométrico de hardware
    return WatchShape(
```

```

builder: (BuildContext context, WearShape shape, Widget? child) {

  // 2. Inyección del listener del ciclo de vida energético del reloj
  return AmbientMode(
    builder: (BuildContext context, WearMode mode, Widget? child) {

      // Abstracción de estados para evaluación lógica en los widgets
      final bool isAmbient = mode == WearMode.ambient;
      final bool isRound = shape == WearShape.round;

      return Scaffold(
        backgroundColor: Colors.black, // Doble protección de contraste
        energético
        body: Center(
          child: Container(
            // CONTROL DE AJUSTE GEOMÉTRICO (Mitigación de Clipping):
            // Si el hardware detectado es circular, empujamos la interfaz
            un 20% hacia adentro.
            // Si es cuadrado, usamos un margen estándar de 8 píxeles.
            padding: EdgeInsets.all(isRound ? 20.0 : 8.0),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              crossAxisAlignment: CrossAxisAlignment.center,
              children: [
                // Renderizado reactivo del icono según el estado de la
                pantalla
                Icon(
                  isAmbient ? Icons.watch_distribute_sharp :
                  Icons.smartwatch,

                  size: 38,
                  color: isAmbient ? Colors.grey : Colors.cyanAccent,
                ),
                const SizedBox(height: 8),

                const Text(
                  'WEAR OS LAB',
                  textAlign: TextAlign.center,
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 16,
                    fontWeight: FontWeight.bold,
                    letterSpacing: 1.1,
                  ),
                ),
                const SizedBox(height: 4),

                Text(
                  isAmbient ? 'MODO: BAJO CONSUMO' : 'MODO: HARDWARE
                  ACTIVO',

                  style: TextStyle(
                    color: isAmbient ? Colors.grey : Colors.cyan,
                    fontSize: 9,
                    fontFamily: 'monospace',
                  ),
                ),
              ],
            ),
          ),
        ),
      );
    },
  );
}

```

```

    ),
    const SizedBox(height: 12),

    // ARQUITECTURA DE INTERACCIÓN PASIVA:
    // Ocultamos componentes interactivos pesados si el sensor
táctil reduce su tasa de muestreo en Modo Ambiente.
    if (!isAmbient)
      ElevatedButton.icon(
        onPressed: () {
          _showWatchAlert(context);
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.cyanAccent.shade700,
          foregroundColor: Colors.white,
          padding: const EdgeInsets.symmetric(horizontal: 10,
vertical: 2),
          shape: const StadiumBorder(), // Factor de forma de
píldora para pantallas curvas
        ),
        icon: const Icon(Icons.touch_app, size: 12),
        label: const Text('Probar', style: TextStyle(fontSize:
11)),
      ),
    ],
  ),
),
);
},
);
};
},
);
}

// Notificación asíncrona optimizada para la escala del wearable
void _showWatchAlert(BuildContext context) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      duration: Duration(milliseconds: 800),
      backgroundColor: Colors.white30,
      content: Text(
        'Interrupción Táctil Procesada',
        textAlign: TextAlign.center,
        style: TextStyle(fontSize: 10, fontWeight: FontWeight.bold),
      ),
    ),
  );
}
}

```

4. Guía Didáctica

- **Velocidad de Compilación:** Nota la tremenda diferencia en tiempos al ejecutar `flutter run` en este proyecto en comparación con el anterior. Al no tener dependencias cruzadas de plugins telefónicos complejos, Gradle compila e inyecta el código en el reloj de forma casi instantánea.
- **Simulación del Ciclo de Vida Independiente:** Al aislar la aplicación, pueden realizar pruebas unitarias del comportamiento de la pantalla (Pasar de Activo a Ambiente) de manera limpia mediante la consola de ADB o los controles directos del emulador de Wear OS, simulando el comportamiento real de la muñeca del usuario.